[TÊN TRƯỜNG] (ĐẠI HỌC BÁCH KHOA HÀ NỘI)

[KHOA / VIỆN] (VIỆN TOÁN ỨNG DỤNG VÀ TIN HỌC - FAMI)

BÁO CÁO BÀI TẬP 2 (ASSIGNMENT 2)

NGHIÊN CỨU VÀ LẬP TRÌNH MIPS VÀ RISC-V INSTRUCTION SET ARCHITECTURES

Học phần:	[TÊN HỌC PHẦN] (Kiến trúc máy tính)
Mã học phần:	[MÃ HỌC PHẦN]
Giảng viên hướng dẫn:	[GIẢNG VIÊN]
Sinh viên thực hiện:	Nguyễn Phúc Vinh
Mã số sinh viên (MSSV):	202419019
Lớp:	[LỚP]
Mã nguồn GitHub:	https://github.com/latdat/KTMT_BT_02

HÀ NỘI, NĂM HỌC 2025

MỤC LỤC BÁO CÁO

PHẦN 1: TỔNG QUAN, MỤC TIÊU VÀ HƯỚNG DẪN CÀI ĐẶT

PHẦN 2: NGHIÊN CỨU VÀ SO SÁNH MIPS & RISC-V ISA (TASK 1)

PHẦN 3: CHƯƠNG TRÌNH TÍNH TOÁN SỐ HỌC CƠ BẢN (TASK 2 & 4)

PHẦN 4: CHƯƠNG TRÌNH NÂNG CAO - CÁC CẤU TRÚC RA QUYẾT ĐỊNH (TASK 3 & 4)

PHẦN 5: KIỂM THỬ VÀ ĐÁNH GIÁ KẾT QUẢ (TESTING & EVALUATION)

PHẦN 6: KẾT LUẬN VÀ TÀI LIỆU THAM KHẢO

PHẦN 1: TỔNG QUAN, MỤC TIÊU VÀ HƯỚNG DẪN CÀI ĐẶT

1.1. Thông tin sinh viên và Mục tiêu bài tập

Báo cáo Bài tập 2 được thực hiện bởi sinh viên **Nguyễn Phúc Vinh** (MSSV: **202419019**) theo đề bài trên slide môn học Kiến trúc máy tính (FAMI-HUST). Toàn bộ mã nguồn, cấu hình và kịch bản kiểm thử được quản lý công khai tại repository: `https://github.com/latdat/KTMT_BT_02`.

Nội dung bài tập bao gồm 5 nhiệm vụ chính:

1. **Task 1 (Investigate ISA):** Nghiên cứu chuyên sâu về kiến trúc tập lệnh MIPS và RISC-V, phân tích cấu trúc thanh ghi, tập lệnh và cơ chế gọi hệ thống.
2. **Task 2 (MIPS Basic):** Viết và chạy chương trình MIPS nhập 2 số nguyên, thực hiện 4 phép toán: Cộng (+), Trừ (-), Nhân (*), Chia (/), có xử lý ngoại lệ chia cho 0.
3. **Task 3 (MIPS Advanced - Decision Making):** Nâng cấp chương trình MIPS minh họa các cấu trúc ra quyết định: `if-then-else`, vòng lặp `loop`, vòng lặp `while`, và cấu trúc rẽ nhánh tương đương `switch/case`.
4. **Task 4 (Convert to RISC-V):** Chuyển đổi toàn bộ các chương trình MIPS sang kiến trúc tập lệnh RISC-V (RV32IM) chạy trên trình mô phỏng RARS.
5. **Task 5 (Report & Submission):** Tổng hợp kết quả thực nghiệm, so sánh đối chiếu hai kiến trúc và xuất báo cáo nộp bài theo quy chuẩn đặt tên: `NguyenPhucVinh_A2.PDF`.

1.2. Hướng dẫn cài đặt và môi trường thực thi

1. Môi trường Java (JRE/JDK)

Cả MARS và RARS đều được đóng gói dưới dạng tệp thực thi Java (`.jar`). Kiểm tra cài đặt Java trong terminal:

```
java -version
```

Nếu chưa có Java, người dùng tải và cài đặt Java (như Eclipse Adoptium Temurin hoặc Oracle JDK) và cấu hình biến môi trường PATH.

2. Cài đặt và sử dụng MARS (MIPS Simulator)

1. Tải tệp `Mars.jar` từ trang chủ chính thức: Missouri State University CS MARS.
2. Khởi động bằng lệnh: `java -jar Mars.jar`.

3. Mở tệp mã nguồn MIPS (`mips/basic_calculator.asm` hoặc `mips/advanced_calculator.asm`).
4. Nhấn **F3** (Assemble) để dịch mã sang mã máy, sau đó nhấn **F5** (Go) để thực thi. Tương tác nhập xuất tại thẻ **Run I/O**.

3. Cài đặt và sử dụng RARS (RISC-V Simulator)

1. Tải tệp `rars.jar` từ kho phát hành GitHub: GitHub RARS Releases.
2. Khởi động bằng lệnh: `java -jar rars.jar`.
3. Mở tệp RISC-V (`riscv/basic_calculator.s` hoặc `riscv/advanced_calculator.s`) → Nhấn **F3** (Assemble) → Nhấn **F5** (Go) và quan sát kết quả tại màn hình **Run I/O**.

PHẦN 2: NGHIÊN CỨU VÀ SO SÁNH MIPS & RISC-V ISA (TASK 1)

2.1. Tập thanh ghi và Quy ước sử dụng (ABI)

Cả MIPS và RISC-V đều là kiến trúc RISC 32-bit theo mô hình **Load/Store**: chỉ thực hiện tính toán trên các thanh ghi bên trong CPU và truy xuất bộ nhớ thông qua các lệnh nạp/ghi riêng biệt.

Nhóm thanh ghi	MIPS ($0 - $31)	RISC-V (x0 - x31)	Mô tả chức năng theo chuẩn ABI
Hằng số 0	`$zero` ($0)	`zero` (x0)	Luôn mang giá trị cố định bằng 0 ở mức phần cứng.
Địa chỉ trả về	`$ra` ($31)	`ra` (x1)	Lưu địa chỉ quay về sau lời gọi hàm (jal / jalr).
Con trỏ ngăn xếp	`$sp` ($29)	`sp` (x2)	Stack Pointer quản lý đỉnh khung ngăn xếp.
Tham số & Trả về	`$v0-$v1` (trả về) `$a0-$a3` (tham số)	`a0-a1` (trả về) `a0-a7` (tham số)	Truyền tham số đầu vào và nhận giá trị trả về của hàm.
Thanh ghi tạm (Temp)	`$t0-$t9`	`t0-t6`	Lưu trữ tạm thời, không cần bảo toàn qua lời gọi hàm.
Thanh ghi lưu trữ (Saved)	`$s0-$s7`	`s0-s11`	Phải được hàm con lưu lại ngăn xếp nếu có sửa đổi.
Thanh ghi Nhân/Chia	Cặp `HI` và `LO`	**Không có**	MIPS dùng HI/LO; RISC-V ghi trực tiếp vào thanh ghi đích.

Bảng 1: So sánh quy ước thanh ghi giữa MIPS và RISC-V

2.2. So sánh tập lệnh và gọi hệ thống (Syscall vs Ecall)

- **Phép toán nhân và chia:** Trong MIPS, lệnh `mult` và `div` tự động đặt kết quả vào cặp thanh ghi `HI:LO`. Để đọc kết quả ra thanh ghi đa dụng, bắt buộc phải dùng thêm lệnh `mflo` hoặc `mfhi`. Ngược lại, kiến trúc mở rộng M của RISC-V cung cấp các lệnh `mul`, `div` ghi trực tiếp thương/tích vào thanh ghi đích được chỉ định.

- **Cơ chế gọi hệ thống:** MIPS (MARS) dùng lệnh `syscall` với mã dịch vụ nằm trong `$v0`. RISC-V (RARS) dùng lệnh `ecall` với mã dịch vụ nằm trong `a7` và dữ liệu trả về nằm trong `a0`.

Thao tác	Cú pháp MIPS	Cú pháp RISC-V (RV32IM)
Cộng 2 số	`add $t2, $t0, $t1`	`add t2, t0, t1`
Trừ 2 số	`sub $t3, $t0, $t1`	`sub t3, t0, t1`
Nhân 2 số	`mult $t0, $t1; mflo $t4`	`mul t4, t0, t1`
Chia 2 số	`div $t0, $t1; mflo $t5`	`div t5, t0, t1`
So sánh bằng	`beq $t0, $t1, label`	`beq t0, t1, label`
So sánh với 0	`beq $t1, $zero, label`	`beqz t1, label`
So sánh lớn hơn	`bgt $t0, $t1, label`	`bgt t0, t1, label`
Nhảy không điều kiện	`j label`	`j label`
In chuỗi ký tự	`li $v0, 4; la $a0, msg; syscall`	`li a7, 4; la a0, msg; ecall`
Nhập số nguyên	`li $v0, 5; syscall; move $t0, $v0`	`li a7, 5; ecall; mv t0, a0`
Thoát chương trình	`li $v0, 10; syscall`	`li a7, 10; ecall`

Bảng 2: Ánh xạ tập lệnh tiêu biểu giữa MIPS và RISC-V

PHẦN 3: CHƯƠNG TRÌNH TÍNH TOÁN SỐ HỌC CƠ BẢN (TASK 2 & 4)

3.1. Ý tưởng thiết kế và Xử lý ngoại lệ chia cho 0

Chương trình cơ bản nhận vào 2 số nguyên a và b từ bàn phím, thực hiện tính toán và in ra đầy đủ 4 kết quả: Cộng, Trừ, Nhân, Chia. Để ngăn chặn lỗi crash hệ thống khi $b = 0$, chương trình kiểm tra điều kiện bằng lệnh rẽ nhánh (`beq $t1, $zero` trên MIPS hoặc `beqz t1` trên RISC-V). Nếu $b = 0$, luồng thực thi bỏ qua lệnh chia và in thông báo lỗi *"Cannot divide by zero"* an toàn.

3.2. Mã nguồn MIPS cơ bản (mips/basic_calculator.asm)

```
# ================================================================================
# Program: basic_calculator.asm
# Description: MIPS Basic Integer Calculator (Add, Sub, Mul, Div)
# Author: Nguyễn Phúc Vinh (MSSV: 202419019)
# Target Simulator: MARS (MIPS Assembler and Runtime Simulator)
#
# Register Allocation:
#   $v0 - Syscall service selector / integer return value
#   $a0 - Syscall argument (integer to print / string address to print)
#   $t0 - First input integer (a)
#   $t1 - Second input integer (b)
#   $t2 - Addition result (a + b)
#   $t3 - Subtraction result (a - b)
#   $t4 - Multiplication result (a * b)
#   $t5 - Division quotient (a / b)
#
# Syscall Services Used:
#   1  - print_int ($a0 = integer to print)
#   4  - print_string ($a0 = address of null-terminated string)
#   5  - read_int (returns value in $v0)
#   10 - exit (terminate program execution)
# ================================================================================

.data
    # Prompts and headers
    header_msg:     .asciiz "========================================\n       MIPS BAS
IC CALCULATOR\n========================================\n"
    prompt_first:   .asciiz "Enter first integer: "
    prompt_second:  .asciiz "Enter second integer: "
    newline:        .asciiz "\n"

    # Result messages
    msg_add:        .asciiz "\nAddition: "
    msg_sub:        .asciiz "Subtraction: "
    msg_mul:        .asciiz "Multiplication: "
    msg_div:        .asciiz "Division: "
```

```
    msg_div_zero:   .asciiz "Cannot divide by zero\n"

.text
.globl main

main:
    # ------------------------------------------------------------------------
    # Step 1: Print title / header
    # ------------------------------------------------------------------------
    li   $v0, 4                # Syscall 4: print_string
    la   $a0, header_msg       # Load address of header_msg into $a0
    syscall

    # ------------------------------------------------------------------------
    # Step 2: Read first integer (a)
    # ------------------------------------------------------------------------
    li   $v0, 4                # Syscall 4: print_string
    la   $a0, prompt_first     # Load address of prompt_first
    syscall

    li   $v0, 5                # Syscall 5: read_int
    syscall
    move $t0, $v0              # Store first integer (a) in $t0

    # ------------------------------------------------------------------------
    # Step 3: Read second integer (b)
    # ------------------------------------------------------------------------
    li   $v0, 4                # Syscall 4: print_string
    la   $a0, prompt_second    # Load address of prompt_second
    syscall

    li   $v0, 5                # Syscall 5: read_int
    syscall
    move $t1, $v0              # Store second integer (b) in $t1

    # ------------------------------------------------------------------------
    # Step 4: Perform Addition: a + b
    # ------------------------------------------------------------------------
    add  $t2, $t0, $t1         # $t2 = $t0 + $t1

    li   $v0, 4                # Syscall 4: print_string
    la   $a0, msg_add          # Print "Addition: "
    syscall

    li   $v0, 1                # Syscall 1: print_int
    move $a0, $t2              # Move addition result into $a0
    syscall

    li   $v0, 4                # Syscall 4: print_string
    la   $a0, newline          # Print newline
    syscall

    # ------------------------------------------------------------------------
    # Step 5: Perform Subtraction: a - b
    # ------------------------------------------------------------------------
    sub  $t3, $t0, $t1         # $t3 = $t0 - $t1
```

```
    li   $v0, 4                # Syscall 4: print_string
    la   $a0, msg_sub          # Print "Subtraction: "
    syscall

    li   $v0, 1                # Syscall 1: print_int
    move $a0, $t3              # Move subtraction result into $a0
    syscall

    li   $v0, 4                # Syscall 4: print_string
    la   $a0, newline          # Print newline
    syscall

    # ------------------------------------------------------------------------
    # Step 6: Perform Multiplication: a * b
    # MIPS architecture stores 64-bit product in HI and LO special registers.
    # We retrieve the lower 32 bits from LO register using mflo.
    # ------------------------------------------------------------------------
    mult $t0, $t1              # HI:LO = $t0 * $t1 (signed 32-bit multiplication)
    mflo $t4                   # $t4 = lower 32 bits from LO register

    li   $v0, 4                # Syscall 4: print_string
    la   $a0, msg_mul          # Print "Multiplication: "
    syscall

    li   $v0, 1                # Syscall 1: print_int
    move $a0, $t4              # Move product into $a0
    syscall

    li   $v0, 4                # Syscall 4: print_string
    la   $a0, newline          # Print newline
    syscall

    # ------------------------------------------------------------------------
    # Step 7: Perform Division: a / b with zero-divisor check
    # Check if second integer $t1 == 0 before executing div instruction.
    # ------------------------------------------------------------------------
    li   $v0, 4                # Syscall 4: print_string
    la   $a0, msg_div          # Print "Division: "
    syscall

    beq  $t1, $zero, div_by_zero # If $t1 == 0, branch to div_by_zero label

    # Divisor is non-zero: perform division
    div  $t0, $t1              # LO = $t0 / $t1 (quotient), HI = $t0 % $t1 (remainde
r)
    mflo $t5                   # $t5 = quotient from LO

    li   $v0, 1                # Syscall 1: print_int
    move $a0, $t5              # Move quotient into $a0
    syscall

    li   $v0, 4                # Syscall 4: print_string
    la   $a0, newline          # Print newline
    syscall

    j    exit_program          # Skip division-by-zero error handler
```

```
div_by_zero:
    li   $v0, 4                # Syscall 4: print_string
    la   $a0, msg_div_zero     # Print "Cannot divide by zero\n"
    syscall

exit_program:
    # ------------------------------------------------------------------------
    # Step 8: Clean program termination
    # ------------------------------------------------------------------------
    li   $v0, 10               # Syscall 10: exit
    syscall
```

3.3. Mã nguồn RISC-V cơ bản (riscv/basic_calculator.s)

```
# ============================================================================
# Program: basic_calculator.s
# Description: RISC-V Basic Integer Calculator (Add, Sub, Mul, Div)
# Author: Nguyễn Phúc Vinh (MSSV: 202419019)
# Target Simulator: RARS (RISC-V Assembler and Runtime Simulator)
# Architecture: RV32IM (RV32I Base Integer + M Extension for mul/div)
#
# Register Allocation (Standard RISC-V ABI Naming):
#   a7 - Environment call (ecall) service code
#   a0 - Syscall argument (integer / address) & return value for read_int
#   t0 - First input integer (a)
#   t1 - Second input integer (b)
#   t2 - Addition result (a + b)
#   t3 - Subtraction result (a - b)
#   t4 - Multiplication result (a * b)
#   t5 - Division quotient (a / b)
#
# RARS Environment Calls (ecall):
#   Service 1  (print_int)    : a7 = 1,  a0 = integer value to print
#   Service 4  (print_string) : a7 = 4,  a0 = address of null-terminated string
#   Service 5  (read_int)     : a7 = 5,  returns integer value in a0
#   Service 10 (exit)         : a7 = 10, terminates program execution
#
# Key Architectural Differences vs. MIPS:
#   1. Syscall / Ecall Service Register:
#      - MIPS uses $v0 for service selection.
#      - RISC-V uses a7 for service selection.
#   2. Multiplication & Division:
#      - MIPS: uses mult/div with special HI/LO registers (accessed via mfhi/mflo).
#      - RISC-V (M Extension): mul/div write result directly to a general register.
#   3. Register Prefixes:
#      - MIPS registers typically prefixed with '$' ($t0, $v0).
#      - RISC-V registers omit the '$' (t0, a7).
# ============================================================================

.data
    # Prompts and headers
    header_msg:     .asciz "========================================\n      RISC-V BAS
IC CALCULATOR\n========================================\n"
    prompt_first:   .asciz "Enter first integer: "
    prompt_second:  .asciz "Enter second integer: "
```

```
    newline:        .asciz "\n"

    # Result messages
    msg_add:        .asciz "\nAddition: "
    msg_sub:        .asciz "Subtraction: "
    msg_mul:        .asciz "Multiplication: "
    msg_div:        .asciz "Division: "
    msg_div_zero:   .asciz "Cannot divide by zero\n"

.text
.globl main

main:
    # ------------------------------------------------------------------------
    # Step 1: Print title / header
    # MIPS:   li $v0, 4; la $a0, header_msg; syscall
    # RISC-V: li a7, 4;  la a0, header_msg;  ecall
    # ------------------------------------------------------------------------
    li   a7, 4                  # ecall service 4: print_string
    la   a0, header_msg         # Load address of header_msg into a0
    ecall

    # ------------------------------------------------------------------------
    # Step 2: Read first integer (a)
    # ------------------------------------------------------------------------
    li   a7, 4                  # Print prompt
    la   a0, prompt_first
    ecall

    li   a7, 5                  # ecall service 5: read_int
    ecall
    mv   t0, a0                 # In RISC-V, read_int returns in a0; save to t0

    # ------------------------------------------------------------------------
    # Step 3: Read second integer (b)
    # ------------------------------------------------------------------------
    li   a7, 4                  # Print prompt
    la   a0, prompt_second
    ecall

    li   a7, 5                  # ecall service 5: read_int
    ecall
    mv   t1, a0                 # Save second integer to t1

    # ------------------------------------------------------------------------
    # Step 4: Perform Addition: a + b
    # MIPS:   add $t2, $t0, $t1
    # RISC-V: add t2, t0, t1
    # ------------------------------------------------------------------------
    add  t2, t0, t1             # t2 = t0 + t1

    li   a7, 4                  # Print "Addition: "
    la   a0, msg_add
    ecall

    li   a7, 1                  # ecall service 1: print_int
    mv   a0, t2                 # Move addition result into a0
```

```
    ecall

    li   a7, 4                  # Print newline
    la   a0, newline
    ecall

    # ------------------------------------------------------------------------
    # Step 5: Perform Subtraction: a - b
    # MIPS:   sub $t3, $t0, $t1
    # RISC-V: sub t3, t0, t1
    # ------------------------------------------------------------------------
    sub  t3, t0, t1             # t3 = t0 - t1

    li   a7, 4                  # Print "Subtraction: "
    la   a0, msg_sub
    ecall

    li   a7, 1                  # Print integer
    mv   a0, t3
    ecall

    li   a7, 4                  # Print newline
    la   a0, newline
    ecall

    # ------------------------------------------------------------------------
    # Step 6: Perform Multiplication: a * b
    # MIPS:   mult $t0, $t1; mflo $t4
    # RISC-V: mul t4, t0, t1 (Standard 'M' extension directly puts product in t4)
    # ------------------------------------------------------------------------
    mul  t4, t0, t1             # t4 = (t0 * t1)[31:0]

    li   a7, 4                  # Print "Multiplication: "
    la   a0, msg_mul
    ecall

    li   a7, 1                  # Print integer
    mv   a0, t4
    ecall

    li   a7, 4                  # Print newline
    la   a0, newline
    ecall

    # ------------------------------------------------------------------------
    # Step 7: Perform Division: a / b with zero-divisor check
    # MIPS:   beq $t1, $zero, div_by_zero
    # RISC-V: beqz t1, div_by_zero  (or beq t1, zero, div_by_zero)
    # ------------------------------------------------------------------------
    li   a7, 4                  # Print "Division: "
    la   a0, msg_div
    ecall

    beqz t1, div_by_zero        # If t1 == 0, branch to div_by_zero error

    # Non-zero divisor: compute division
    # MIPS:   div $t0, $t1; mflo $t5
```

```
    # RISC-V: div t5, t0, t1 (Standard 'M' extension directly puts quotient in t5)
    div  t5, t0, t1             # t5 = t0 / t1

    li   a7, 1                  # Print integer
    mv   a0, t5
    ecall

    li   a7, 4                  # Print newline
    la   a0, newline
    ecall

    j    exit_program           # Jump to exit

div_by_zero:
    li   a7, 4                  # Print "Cannot divide by zero\n"
    la   a0, msg_div_zero
    ecall

exit_program:
    # ------------------------------------------------------------------------
    # Step 8: Clean program termination
    # MIPS:   li $v0, 10; syscall
    # RISC-V: li a7, 10;  ecall
    # ------------------------------------------------------------------------
    li   a7, 10                 # ecall service 10: exit
    ecall
```

3.4. Kết quả chạy thử nghiệm cơ bản

ẢNH 1 — KẾT QUẢ CHẠY MIPS BASIC TRÊN MARS

(Nhập: a = 20, b = 5 → In ra: Addition: 25, Subtraction: 15, Multiplication: 100, Division: 4)

Hình 1: Kết quả thực hiện các phép toán số nguyên bằng MIPS trên MARS

ẢNH 4 — KẾT QUẢ CHẠY RISC-V BASIC TRÊN RARS

(Chạy tệp riscv/basic_calculator.s trên RARS với a = 20, b = 5 chứng minh chuyển đổi thành công)

Hình 2: Kết quả chương trình số học RISC-V trên RARS

ẢNH 6 — KIỂM TRA CHIA CHO 0 (EDGE CASE)

(Nhập: a = 10, b = 0 → In thông báo: Division: Cannot divide by zero an toàn không crash)

Hình 3: Minh chứng xử lý an toàn ngoại lệ chia cho 0

PHẦN 4: CHƯƠNG TRÌNH NÂNG CAO - CÁC CẤU TRÚC RA QUYẾT ĐỊNH (TASK 3 & 4)

4.1. Thiết kế hệ thống Menu tương tác

Chương trình nâng cao tích hợp đầy đủ các cấu trúc ra quyết định theo yêu cầu của Task 3:

- **Cấu trúc While:** Menu lặp lại liên tục sau mỗi lượt tính toán cho tới khi người dùng nhập số 7 (Exit). Triển khai bằng nhãn `menu_loop` và lệnh nhảy không điều kiện `j menu_loop`.
- **Cấu trúc Switch/Case:** So sánh giá trị lựa chọn trong thanh ghi (`$s0` / `s0`) với các hằng số 1..7 bằng chuỗi lệnh `beq` để điều phối đến các hàm chức năng. Nếu nhập ngoài khoảng 1..7, rơi vào nhánh `default_case` báo lỗi và quay lại menu.
- **Cấu trúc If / Else-If / Else:** Nằm tại Chức năng 5 (Compare two numbers). So sánh 2 số a, b:
  - $a == b$: In *"Two numbers are equal"*
  - $a > b$: In *"First number is greater"*
  - $a < b$: In *"Second number is greater"*
- **Cấu trúc Loop (Đếm):** Nằm tại Chức năng 6. Nhập số nguyên dương N. Nếu $N \le 0$ thì báo lỗi; nếu $N > 0$ thì dùng biến đếm chạy từ 1 đến N in dãy số `1 2 3 ... N`.

4.2. Mã nguồn MIPS nâng cao (mips/advanced_calculator.asm)

```
# ================================================================================
# Program: advanced_calculator.asm
# Description: MIPS Advanced Integer Calculator with Control Flow
#              Demonstrates: Switch/Case, If/Else-If/Else, While Loop, For Loop,
#              Division-by-zero protection, and Input Validation.
# Author: Nguyễn Phúc Vinh (MSSV: 202419019)
# Target Simulator: MARS (MIPS Assembler and Runtime Simulator)
#
# Register Allocation:
#   $v0 - Syscall service selector / integer input value
#   $a0 - Syscall argument (integer / address)
#   $s0 - User's menu choice
#   $t0 - First operand (a) or general temporary register
#   $t1 - Second operand (b) or loop boundary (N)
#   $t2 - Loop counter / intermediate result
#   $t3 - Arithmetic result (Add / Sub / Mul / Div quotient)
#
# Syscall Services:
#   1  - print_int ($a0 = value)
#   4  - print_string ($a0 = string address)
```

```
#   5  - read_int (result in $v0)
#   10 - exit
# ============================================================================

.data
    # ------------------------------------------------------------------------
    # Menu and Prompts
    # ------------------------------------------------------------------------
    menu_banner:
        .asciiz "\n==============================\n    MIPS INTEGER CALCULATOR\n====
============================\n1. Add\n2. Subtract\n3. Multiply\n4. Divide\n5. Compare
two numbers\n6. Demonstrate loop\n7. Exit\nChoose an option: "

    prompt_a:       .asciiz "Enter first integer (a): "
    prompt_b:       .asciiz "Enter second integer (b): "
    prompt_n:       .asciiz "Enter positive integer N: "

    # ------------------------------------------------------------------------
    # Result and Informational Messages
    # ------------------------------------------------------------------------
    msg_res_add:    .asciiz "Result (a + b) = "
    msg_res_sub:    .asciiz "Result (a - b) = "
    msg_res_mul:    .asciiz "Result (a * b) = "
    msg_res_div:    .asciiz "Result (a / b) = "

    msg_div_zero:   .asciiz "Error: division by zero. Cannot divide by 0.\n"
    msg_invalid:    .asciiz "Invalid option. Please try again.\n"
    msg_exit:       .asciiz "Exiting program. Goodbye!\n"

    # Comparison messages (If-Else-If-Else)
    msg_cmp_gt:     .asciiz "First number is greater\n"
    msg_cmp_lt:     .asciiz "Second number is greater\n"
    msg_cmp_eq:     .asciiz "Two numbers are equal\n"

    # Loop demonstration messages
    msg_loop_start: .asciiz "Loop sequence from 1 to N:\n"
    msg_loop_err:   .asciiz "Error: N must be greater than 0.\n"
    space:          .asciiz " "
    newline:        .asciiz "\n"

.text
.globl main

main:

# ============================================================================
# STRUCTURE C: WHILE LOOP (Menu Driver)
# High-Level Logic:
#   while (choice != 7) {
#       displayMenu();
#       choice = readChoice();
#       executeChoice(choice);
#   }
# Implemented using a loop label and an unconditional jump back to the start.
# ============================================================================
menu_loop:
    # 1. Display Menu Banner
```

```
    li   $v0, 4
    la   $a0, menu_banner
    syscall

    # 2. Read User's Choice
    li   $v0, 5
    syscall
    move $s0, $v0               # Save user choice in $s0

# ================================================================================
# STRUCTURE A: SWITCH / CASE EQUIVALENT
# High-Level Logic:
#   switch(choice) {
#       case 1: do_add();      break;
#       case 2: do_sub();      break;
#       case 3: do_mul();      break;
#       case 4: do_div();      break;
#       case 5: do_compare();  break;
#       case 6: do_loop();     break;
#       case 7: do_exit();     break;
#       default: do_invalid(); break;
#   }
# In MIPS Assembly, switch/case is implemented via sequential conditional branches
# (beq) comparing the choice register against constant literals, jumping to handler
# labels, with a fall-through to the default label.
# ================================================================================
case_1:
    li   $t0, 1
    beq  $s0, $t0, do_add

case_2:
    li   $t0, 2
    beq  $s0, $t0, do_subtract

case_3:
    li   $t0, 3
    beq  $s0, $t0, do_multiply

case_4:
    li   $t0, 4
    beq  $s0, $t0, do_divide

case_5:
    li   $t0, 5
    beq  $s0, $t0, do_compare

case_6:
    li   $t0, 6
    beq  $s0, $t0, do_loop

case_7:
    li   $t0, 7
    beq  $s0, $t0, do_exit

default_case:
    # --------------------------------------------------------------------------
    # STRUCTURE F: INVALID OPTION HANDLER
```

```
    # ---------------------------------------------------------------------
    li   $v0, 4
    la   $a0, msg_invalid
    syscall
    j    menu_loop              # Return to menu loop

# ============================================================================
# OPTION 1: ADDITION (a + b)
# ============================================================================
do_add:
    # Read first integer
    li   $v0, 4
    la   $a0, prompt_a
    syscall
    li   $v0, 5
    syscall
    move $t0, $v0

    # Read second integer
    li   $v0, 4
    la   $a0, prompt_b
    syscall
    li   $v0, 5
    syscall
    move $t1, $v0

    # Compute sum
    add  $t3, $t0, $t1

    # Print result
    li   $v0, 4
    la   $a0, msg_res_add
    syscall
    li   $v0, 1
    move $a0, $t3
    syscall
    li   $v0, 4
    la   $a0, newline
    syscall

    j    menu_loop              # Break and return to while loop

# ============================================================================
# OPTION 2: SUBTRACTION (a - b)
# ============================================================================
do_subtract:
    # Read first integer
    li   $v0, 4
    la   $a0, prompt_a
    syscall
    li   $v0, 5
    syscall
    move $t0, $v0

    # Read second integer
    li   $v0, 4
    la   $a0, prompt_b
```

```
    syscall
    li   $v0, 5
    syscall
    move $t1, $v0

    # Compute difference
    sub  $t3, $t0, $t1

    # Print result
    li   $v0, 4
    la   $a0, msg_res_sub
    syscall
    li   $v0, 1
    move $a0, $t3
    syscall
    li   $v0, 4
    la   $a0, newline
    syscall

    j    menu_loop              # Break and return to while loop

# ================================================================================
# OPTION 3: MULTIPLICATION (a * b)
# ================================================================================
do_multiply:
    # Read first integer
    li   $v0, 4
    la   $a0, prompt_a
    syscall
    li   $v0, 5
    syscall
    move $t0, $v0

    # Read second integer
    li   $v0, 4
    la   $a0, prompt_b
    syscall
    li   $v0, 5
    syscall
    move $t1, $v0

    # Multiply: result stored in HI:LO special registers
    mult $t0, $t1
    mflo $t3                    # Retrieve lower 32-bit product

    # Print result
    li   $v0, 4
    la   $a0, msg_res_mul
    syscall
    li   $v0, 1
    move $a0, $t3
    syscall
    li   $v0, 4
    la   $a0, newline
    syscall

    j    menu_loop              # Break and return to while loop
```

```
# ============================================================================
# OPTION 4: DIVISION (a / b) with DIVISION-BY-ZERO PROTECTION (Structure E)
# ============================================================================
do_divide:
    # Read first integer (dividend)
    li   $v0, 4
    la   $a0, prompt_a
    syscall
    li   $v0, 5
    syscall
    move $t0, $v0

    # Read second integer (divisor)
    li   $v0, 4
    la   $a0, prompt_b
    syscall
    li   $v0, 5
    syscall
    move $t1, $v0

    # ----------------------------------------------------------------------
    # STRUCTURE E: Check if divisor ($t1) == 0 before performing division
    # ----------------------------------------------------------------------
    beq  $t1, $zero, div_zero_error

    # Divisor != 0: Safe to divide
    div  $t0, $t1
    mflo $t3                    # Quotient stored in LO register

    # Print result
    li   $v0, 4
    la   $a0, msg_res_div
    syscall
    li   $v0, 1
    move $a0, $t3
    syscall
    li   $v0, 4
    la   $a0, newline
    syscall

    j    menu_loop              # Return to menu loop

div_zero_error:
    li   $v0, 4
    la   $a0, msg_div_zero      # Print error message
    syscall
    j    menu_loop              # Return to menu loop without crashing

# ============================================================================
# STRUCTURE B: IF / ELSE-IF / ELSE (Compare Two Numbers)
# High-Level Logic:
#   if (a > b) {
#       print("First number is greater");
#   } else if (a < b) {
#       print("Second number is greater");
#   } else {
```

```
#       print("Two numbers are equal");
#   }
# ================================================================================
do_compare:
    # Read first integer
    li   $v0, 4
    la   $a0, prompt_a
    syscall
    li   $v0, 5
    syscall
    move $t0, $v0

    # Read second integer
    li   $v0, 4
    la   $a0, prompt_b
    syscall
    li   $v0, 5
    syscall
    move $t1, $v0

    # Branch Condition 1: if (a == b) -> branch to equal
    beq  $t0, $t1, cmp_equal

    # Branch Condition 2: if (a > b) -> branch to greater
    # In MIPS, bgt $t0, $t1, cmp_greater (pseudo-instruction)
    bgt  $t0, $t1, cmp_greater

    # Else Condition: a < b
    li   $v0, 4
    la   $a0, msg_cmp_lt        # "Second number is greater"
    syscall
    j    menu_loop

cmp_greater:
    li   $v0, 4
    la   $a0, msg_cmp_gt        # "First number is greater"
    syscall
    j    menu_loop

cmp_equal:
    li   $v0, 4
    la   $a0, msg_cmp_eq        # "Two numbers are equal"
    syscall
    j    menu_loop

# ================================================================================
# STRUCTURE D: COUNTED LOOP (Demonstrate Loop)
# High-Level Logic:
#   if (N <= 0) {
#       print("Error: N must be greater than 0");
#   } else {
#       for (counter = 1; counter <= N; counter++) {
#           print(counter);
#           print(" ");
#       }
#       print("\n");
#   }
```

```
# ============================================================================
do_loop:
    # Read N
    li   $v0, 4
    la   $a0, prompt_n
    syscall
    li   $v0, 5
    syscall
    move $t1, $v0               # $t1 = N (upper bound)

    # Check if N <= 0 (Input Validation)
    blez $t1, loop_invalid_n    # If $t1 <= 0, branch to error

    # Valid N > 0: Initialize counter = 1
    li   $t2, 1                 # $t2 = loop counter = 1

    li   $v0, 4
    la   $a0, msg_loop_start
    syscall

loop_body:
    # Check termination condition: if counter > N, exit loop
    bgt  $t2, $t1, loop_done

    # Print current counter value
    li   $v0, 1
    move $a0, $t2
    syscall

    # Print separator space
    li   $v0, 4
    la   $a0, space
    syscall

    # Increment counter: counter = counter + 1
    addi $t2, $t2, 1

    # Jump to next iteration
    j    loop_body

loop_done:
    li   $v0, 4
    la   $a0, newline
    syscall
    j    menu_loop              # Return to menu loop

loop_invalid_n:
    li   $v0, 4
    la   $a0, msg_loop_err
    syscall
    j    menu_loop              # Return to menu loop

# ============================================================================
# EXIT PROGRAM
# ============================================================================
do_exit:
    li   $v0, 4
```

```
    la   $a0, msg_exit
    syscall

    li   $v0, 10                # Syscall 10: Exit program
    syscall
```

4.3. Mã nguồn RISC-V nâng cao (riscv/advanced_calculator.s)

```
# ================================================================================
# Program: advanced_calculator.s
# Description: RISC-V Advanced Integer Calculator with Control Flow
#              Demonstrates: Switch/Case, If/Else-If/Else, While Loop, For Loop,
#              Division-by-zero protection, and Input Validation.
# Author: Nguyễn Phúc Vinh (MSSV: 202419019)
# Target Simulator: RARS (RISC-V Assembler and Runtime Simulator)
# Architecture: RV32IM (RV32I Base Integer + M Extension for mul/div)
#
# Register Allocation (Standard RISC-V ABI Naming):
#   a7 - ecall service code
#   a0 - ecall argument & integer return value
#   s0 - User's menu choice
#   t0 - First operand (a) or general temporary register
#   t1 - Second operand (b) or loop boundary (N)
#   t2 - Loop counter / intermediate register
#   t3 - Arithmetic result (Add / Sub / Mul / Div quotient)
#
# RARS Environment Calls (ecall):
#   1  - print_int ($a0 in MIPS -> a0 in RISC-V)
#   4  - print_string ($a0 in MIPS -> a0 in RISC-V)
#   5  - read_int ($v0 in MIPS -> a0 in RISC-V)
#   10 - exit
# ================================================================================

.data
    # --------------------------------------------------------------------------
    # Menu and Prompts
    # --------------------------------------------------------------------------
    menu_banner:
        .asciz "\n================================\n   RISC-V INTEGER CALCULATOR\n====
============================\n1. Add\n2. Subtract\n3. Multiply\n4. Divide\n5. Compare
two numbers\n6. Demonstrate loop\n7. Exit\nChoose an option: "

    prompt_a:       .asciz "Enter first integer (a): "
    prompt_b:       .asciz "Enter second integer (b): "
    prompt_n:       .asciz "Enter positive integer N: "

    # --------------------------------------------------------------------------
    # Result and Informational Messages
    # --------------------------------------------------------------------------
    msg_res_add:    .asciz "Result (a + b) = "
    msg_res_sub:    .asciz "Result (a - b) = "
    msg_res_mul:    .asciz "Result (a * b) = "
    msg_res_div:    .asciz "Result (a / b) = "

    msg_div_zero:   .asciz "Error: division by zero. Cannot divide by 0.\n"
    msg_invalid:    .asciz "Invalid option. Please try again.\n"
    msg_exit:       .asciz "Exiting program. Goodbye!\n"

    # Comparison messages (If-Else-If-Else)
    msg_cmp_gt:     .asciz "First number is greater\n"
    msg_cmp_lt:     .asciz "Second number is greater\n"
```

```
    msg_cmp_eq:     .asciz "Two numbers are equal\n"

    # Loop demonstration messages
    msg_loop_start: .asciz "Loop sequence from 1 to N:\n"
    msg_loop_err:   .asciz "Error: N must be greater than 0.\n"
    space:          .asciz " "
    newline:        .asciz "\n"

.text
.globl main

main:

# ================================================================================
# STRUCTURE C: WHILE LOOP (Menu Driver)
# High-Level Logic:
#   while (choice != 7) {
#       displayMenu();
#       choice = readChoice();
#       executeChoice(choice);
#   }
# Implemented using a loop label and an unconditional jump (j menu_loop) back.
#
# Contrast with MIPS:
#   MIPS:   j menu_loop
#   RISC-V: j menu_loop  (pseudo-instruction for: jal zero, menu_loop)
# ================================================================================
menu_loop:
    # 1. Display Menu Banner
    # MIPS:   li $v0, 4; la $a0, menu_banner; syscall
    # RISC-V: li a7, 4;  la a0, menu_banner;  ecall
    li   a7, 4
    la   a0, menu_banner
    ecall

    # 2. Read User's Choice
    # MIPS:   li $v0, 5; syscall; move $s0, $v0
    # RISC-V: li a7, 5;  ecall;   mv   s0, a0
    li   a7, 5
    ecall
    mv   s0, a0                 # Save user choice in s0

# ================================================================================
# STRUCTURE A: SWITCH / CASE EQUIVALENT
# High-Level Logic:
#   switch(choice) {
#       case 1: do_add();      break;
#       case 2: do_sub();      break;
#       case 3: do_mul();      break;
#       case 4: do_div();      break;
#       case 5: do_compare();  break;
#       case 6: do_loop();     break;
#       case 7: do_exit();     break;
#       default: do_invalid(); break;
#   }
# In RISC-V, just as in MIPS, switch/case is implemented via sequential conditional
# branches comparing choice register against constant literals.
```

```
# ============================================================================
case_1:
    li   t0, 1
    beq  s0, t0, do_add

case_2:
    li   t0, 2
    beq  s0, t0, do_subtract

case_3:
    li   t0, 3
    beq  s0, t0, do_multiply

case_4:
    li   t0, 4
    beq  s0, t0, do_divide

case_5:
    li   t0, 5
    beq  s0, t0, do_compare

case_6:
    li   t0, 6
    beq  s0, t0, do_loop

case_7:
    li   t0, 7
    beq  s0, t0, do_exit

default_case:
    # ------------------------------------------------------------------------
    # STRUCTURE F: INVALID OPTION HANDLER
    # ------------------------------------------------------------------------
    li   a7, 4
    la   a0, msg_invalid
    ecall
    j    menu_loop              # Return to menu loop

# ============================================================================
# OPTION 1: ADDITION (a + b)
# ============================================================================
do_add:
    # Read first integer
    li   a7, 4
    la   a0, prompt_a
    ecall
    li   a7, 5
    ecall
    mv   t0, a0

    # Read second integer
    li   a7, 4
    la   a0, prompt_b
    ecall
    li   a7, 5
    ecall
    mv   t1, a0
```

```
    # Compute sum
    # MIPS:   add $t3, $t0, $t1
    # RISC-V: add t3, t0, t1
    add  t3, t0, t1

    # Print result
    li   a7, 4
    la   a0, msg_res_add
    ecall
    li   a7, 1
    mv   a0, t3
    ecall
    li   a7, 4
    la   a0, newline
    ecall

    j    menu_loop              # Break and return to while loop

# ============================================================================
# OPTION 2: SUBTRACTION (a - b)
# ============================================================================
do_subtract:
    # Read first integer
    li   a7, 4
    la   a0, prompt_a
    ecall
    li   a7, 5
    ecall
    mv   t0, a0

    # Read second integer
    li   a7, 4
    la   a0, prompt_b
    ecall
    li   a7, 5
    ecall
    mv   t1, a0

    # Compute difference
    # MIPS:   sub $t3, $t0, $t1
    # RISC-V: sub t3, t0, t1
    sub  t3, t0, t1

    # Print result
    li   a7, 4
    la   a0, msg_res_sub
    ecall
    li   a7, 1
    mv   a0, t3
    ecall
    li   a7, 4
    la   a0, newline
    ecall

    j    menu_loop              # Break and return to while loop
```

```
# ================================================================================
# OPTION 3: MULTIPLICATION (a * b)
# ================================================================================
do_multiply:
    # Read first integer
    li   a7, 4
    la   a0, prompt_a
    ecall
    li   a7, 5
    ecall
    mv   t0, a0

    # Read second integer
    li   a7, 4
    la   a0, prompt_b
    ecall
    li   a7, 5
    ecall
    mv   t1, a0

    # Multiply:
    # MIPS:   mult $t0, $t1; mflo $t3
    # RISC-V: mul t3, t0, t1 (Standard 'M' extension writes directly to t3)
    mul  t3, t0, t1

    # Print result
    li   a7, 4
    la   a0, msg_res_mul
    ecall
    li   a7, 1
    mv   a0, t3
    ecall
    li   a7, 4
    la   a0, newline
    ecall

    j    menu_loop              # Break and return to while loop

# ================================================================================
# OPTION 4: DIVISION (a / b) with DIVISION-BY-ZERO PROTECTION (Structure E)
# ================================================================================
do_divide:
    # Read first integer (dividend)
    li   a7, 4
    la   a0, prompt_a
    ecall
    li   a7, 5
    ecall
    mv   t0, a0

    # Read second integer (divisor)
    li   a7, 4
    la   a0, prompt_b
    ecall
    li   a7, 5
    ecall
    mv   t1, a0
```

```
    # ------------------------------------------------------------------------
    # STRUCTURE E: Check if divisor (t1) == 0 before performing division
    # MIPS:   beq  $t1, $zero, div_zero_error
    # RISC-V: beqz t1, div_zero_error  (or beq t1, zero, div_zero_error)
    # ------------------------------------------------------------------------
    beqz t1, div_zero_error

    # Divisor != 0: Safe to divide
    # MIPS:   div $t0, $t1; mflo $t3
    # RISC-V: div t3, t0, t1
    div  t3, t0, t1

    # Print result
    li   a7, 4
    la   a0, msg_res_div
    ecall
    li   a7, 1
    mv   a0, t3
    ecall
    li   a7, 4
    la   a0, newline
    ecall

    j    menu_loop              # Return to menu loop

div_zero_error:
    li   a7, 4
    la   a0, msg_div_zero      # Print error message
    ecall
    j    menu_loop              # Return to menu loop without crashing

# ============================================================================
# STRUCTURE B: IF / ELSE-IF / ELSE (Compare Two Numbers)
# High-Level Logic:
#   if (a == b) {
#       print("Two numbers are equal");
#   } else if (a > b) {
#       print("First number is greater");
#   } else {
#       print("Second number is greater");
#   }
#
# Contrast with MIPS:
#   MIPS:   beq $t0, $t1, label; bgt $t0, $t1, label
#   RISC-V: beq t0, t1, label;   bgt t0, t1, label (pseudo for blt t1, t0, label)
# ============================================================================
do_compare:
    # Read first integer
    li   a7, 4
    la   a0, prompt_a
    ecall
    li   a7, 5
    ecall
    mv   t0, a0

    # Read second integer
```

```
    li   a7, 4
    la   a0, prompt_b
    ecall
    li   a7, 5
    ecall
    mv   t1, a0

    # Branch Condition 1: if (a == b) -> branch to equal
    beq  t0, t1, cmp_equal

    # Branch Condition 2: if (a > b) -> branch to greater
    bgt  t0, t1, cmp_greater

    # Else Condition: a < b
    li   a7, 4
    la   a0, msg_cmp_lt        # "Second number is greater"
    ecall
    j    menu_loop

cmp_greater:
    li   a7, 4
    la   a0, msg_cmp_gt        # "First number is greater"
    ecall
    j    menu_loop

cmp_equal:
    li   a7, 4
    la   a0, msg_cmp_eq        # "Two numbers are equal"
    ecall
    j    menu_loop

# ================================================================================
# STRUCTURE D: COUNTED LOOP (Demonstrate Loop)
# High-Level Logic:
#   if (N <= 0) {
#       print("Error: N must be greater than 0");
#   } else {
#       for (counter = 1; counter <= N; counter++) {
#           print(counter);
#           print(" ");
#       }
#       print("\n");
#   }
#
# Contrast with MIPS:
#   MIPS:   blez $t1, loop_invalid_n
#   RISC-V: blez t1, loop_invalid_n  (pseudo for bge zero, t1, loop_invalid_n)
# ================================================================================
do_loop:
    # Read N
    li   a7, 4
    la   a0, prompt_n
    ecall
    li   a7, 5
    ecall
    mv   t1, a0                # t1 = N (upper bound)
```

```
    # Check if N <= 0 (Input Validation)
    blez t1, loop_invalid_n

    # Valid N > 0: Initialize counter = 1
    li   t2, 1                 # t2 = loop counter = 1

    li   a7, 4
    la   a0, msg_loop_start
    ecall

loop_body:
    # Check termination condition: if counter > N, exit loop
    bgt  t2, t1, loop_done

    # Print current counter value
    li   a7, 1
    mv   a0, t2
    ecall

    # Print separator space
    li   a7, 4
    la   a0, space
    ecall

    # Increment counter: counter = counter + 1
    addi t2, t2, 1

    # Jump to next iteration
    j    loop_body

loop_done:
    li   a7, 4
    la   a0, newline
    ecall
    j    menu_loop             # Return to menu loop

loop_invalid_n:
    li   a7, 4
    la   a0, msg_loop_err
    ecall
    j    menu_loop             # Return to menu loop

# ================================================================================
# EXIT PROGRAM
# ================================================================================
do_exit:
    li   a7, 4
    la   a0, msg_exit
    ecall

    li   a7, 10                # ecall service 10: Exit program
    ecall
```

4.4. Kết quả chạy thử nghiệm nâng cao

ẢNH 2 — MINH HỌA CẤU TRÚC IF--ELSE TRÊN MIPS

(Chọn mục 5: Nhập a = 15, b = 4 → In: First number is greater; hoặc a = 7, b = 7 → In: Two numbers are equal)

Hình 4: Minh họa cấu trúc if--else trong chương trình MIPS trên MARS

ẢNH 3 — MINH HỌA VÒNG LẶP LOOP, WHILE VÀ SWITCH/CASE TRÊN MIPS

(Tại Menu, chọn mục 6 → Nhập N = 5 → In: 1 2 3 4 5 → Menu hiển thị lại để chứng minh while loop)

Hình 5: Minh họa vòng lặp Loop, While và Switch/Case trên MARS

ẢNH 5 — MINH HỌA CHƯƠNG TRÌNH RISC-V NÂNG CAO TRÊN RARS

(Thực thi menu điều khiển và vòng lặp số học trên trình giả lập RARS)

Hình 6: Kết quả chạy chương trình RISC-V nâng cao trên RARS

PHẦN 5: KIỂM THỬ VÀ ĐÁNH GIÁ KẾT QUẢ (TESTING & EVALUATION)

5.1. Ma trận các ca kiểm thử (Test Cases)

ID	Đầu vào	Chức năng	Kết quả kỳ vọng	Kết quả thực tế	Trạng thái
TC01	a = 10, b = 5	Số học cơ bản (+)	Add: 15, Sub: 5, Mul: 50, Div: 2	Addition: 15, Subtraction: 5...	**PASS**
TC02	a = -10, b = 5	Số học (a âm)	Add: -5, Sub: -15, Mul: -50, Div: -2	Addition: -5, Subtraction: -15...	**PASS**
TC03	a = 10, b = -5	Số học (b âm)	Add: 5, Sub: 15, Mul: -50, Div: -2	Addition: 5, Subtraction: 15...	**PASS**
TC04	a = -10, b = -5	Cả hai số âm	Add: -15, Sub: -5, Mul: 50, Div: 2	Addition: -15, Subtraction: -5...	**PASS**
TC05	a = 0, b = 5	Số bị chia bằng 0	Add: 5, Sub: -5, Mul: 0, Div: 0	Addition: 5, Subtraction: -5...	**PASS**
TC06	a = 10, b = 0	Bảo vệ chia cho 0	Division: Cannot divide by zero	Division: Cannot divide by zero	**PASS**
TC07	a = 0, b = 0	Cả hai số bằng 0	Division: Cannot divide by zero	Division: Cannot divide by zero	**PASS**
TC08	a = 7, b = 7	So sánh bằng (if/else)	Two numbers are equal	Two numbers are equal	**PASS**
TC09	a = 15, b = 4	So sánh lớn hơn (if/else)	First number is greater	First number is greater	**PASS**
TC10	a = -3, b = 8	So sánh nhỏ hơn (if/else)	Second number is greater	Second number is greater	**PASS**
TC11	N = 5	Vòng lặp N > 0	Dãy số: 1 2 3 4 5	1 2 3 4 5	**PASS**
TC12	N = 0	Biên vòng lặp N = 0	Error: N must be greater than 0	Error: N must be greater than 0	**PASS**
TC13	N = -5	Biên vòng lặp N < 0	Error: N must be greater than 0	Error: N must be greater than 0	**PASS**

ID	Đầu vào	Chức năng	Kết quả kỳ vọng	Kết quả thực tế	Trạng thái
TC14	Choice = 9	Menu không hợp lệ	Invalid option. Please try again.	Invalid option. Please try again.	**PASS**
TC15	Choice = 7	Thoát ứng dụng	Exiting program. Goodbye!	Exiting program. Goodbye!	**PASS**

Bảng 3: Ma trận kiểm thử xác thực trên MARS và RARS

5.2. Bài học kinh nghiệm và Lỗi thường gặp

- **Tránh ghi đè thanh ghi tham số:** Trong hợp ngữ, các lệnh in ấn `syscall` / `ecall` đều sử dụng thanh ghi `$a0` / `a0`. Do đó, các biến quan trọng như biến đếm vòng lặp hoặc kết quả tạm thời cần được lưu trữ ở các thanh ghi độc lập (như `$t2` hoặc `$s0`) để tránh bị mất dữ liệu khi gọi in ký tự cách hay xuống dòng.
- **Cơ chế rẽ nhánh trực tiếp của RISC-V:** Lệnh `blt`, `bge` của RISC-V trực quan và tiện lợi hơn so với việc phải phối hợp lệnh `slt` và `bne` như trên phần cứng MIPS gốc.

PHẦN 6: KẾT LUẬN VÀ TÀI LIỆU THAM KHẢO

6.1. Kết luận

Bài tập 2 đã hoàn thành trọn vẹn và chuẩn xác 5 yêu cầu cốt lõi theo slide đề bài:

1. Nghiên cứu có hệ thống và đối chiếu sâu sắc giữa hai kiến trúc tập lệnh tiêu biểu MIPS và RISC-V.
2. Xây dựng chương trình MIPS cơ bản thực hiện 4 phép tính số học số nguyên kèm cơ chế xử lý chia cho 0 an toàn.
3. Nâng cấp chương trình MIPS với các cấu trúc ra quyết định: `if-then-else`, vòng lặp `while`, vòng lặp đếm `loop`, và `switch/case`.
4. Chuyển ngữ tương đương 1:1 sang kiến trúc RISC-V (RV32IM) chạy trên trình mô phỏng RARS.
5. Thiết lập bộ ca kiểm thử hoàn chỉnh và tổng hợp toàn bộ kết quả vào báo cáo học thuật.

6.2. Tài liệu tham khảo

1. David A. Patterson, John L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface (RISC-V Edition)*, Morgan Kaufmann Publishers.

2. David A. Patterson, John L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface (MIPS Edition)*, Morgan Kaufmann Publishers.
3. Ken Vollmar, Pete Sanderson, *MARS: MIPS Assembler and Runtime Simulator*, Missouri State University Computer Science.
4. Franz Bettag, *RARS: RISC-V Assembler, Runtime, and Simulator*.